```
function handleCallback(callable $cb): void
{
    // do things then make the callback...
    $cb();
}
```

What we are seeing here is the beginnings of something quite powerful. Functions are sub-units of callable code, true, but here they are also being treated as data. They are values that can be treated like the result of any other expression. Just as an integer variable references an integer value, a function variable references a function.

Functions in PHP can be passed into – and returned by – other functions. This is a neat way of bouncing control around a complex program and treating functions as data opens the door to the "functional programming" paradigm, which we will look at toward the end of this chapter. For now, take a look at the following:

```
$cb = getCallback();
$cb();
getCallback()();

function getCallback(): callable
{
    return function (): void {
        print "Who am I?" . PHP_EOL;
    };
}

Output:
Who am I?
Who am I?
```

This time we're returning an anonymous function, which is perfectly reasonable given that we know functions can be treated like any other data. They even have their own dedicated data type of `callable` as we can see from the declaration `function getCallback(): callable`. We also know that any expression that evaluates to a function can be invoked. We do it by variable assignment: `$cb` is assigned the function reference as its value, then we invoke using `$cb()`.